

Redux & Context API - Complete Notes

Redux is a state management library used mainly with React applications. It helps manage global application state in a predictable way. Context API is React's built-in solution for sharing data between components.

1. What is Redux?

Redux is a centralized state management library. It stores application state inside a single global store. Components can access and update the state from anywhere in the application.

2. Why Redux is Used?

Redux is useful when:

- Many components need shared data
- State becomes difficult to manage
- Applications become large and complex
- Predictable state updates are required

3. Core Concepts of Redux

Concept	Meaning
Store	Central place where state is stored
Action	Object describing what happened
Reducer	Function that updates state
Dispatch	Sends action to reducer
Selector	Reads state from store

4. Redux Working Flow

```
Component
  ↓
Dispatch Action
  ↓
Reducer
  ↓
Store Updated
  ↓
UI Re-renders
```

5. Redux Counter Example

```
npm install redux react-redux @reduxjs/toolkit

// store.js

import { configureStore } from "@reduxjs/toolkit";
import counterReducer from "../counterSlice";

export const store = configureStore({
  reducer: {
    counter: counterReducer
  }
});
```

```

// counterSlice.js
import { createSlice } from "@reduxjs/toolkit";

const counterSlice = createSlice({
  name: "counter",

  initialState: {
    value: 0
  },

  reducers: {

    increment: (state) => {
      state.value++;
    },

    decrement: (state) => {
      state.value--;
    }

  }
});

export const {
  increment,
  decrement
} = counterSlice.actions;

export default counterSlice.reducer;

// App.jsx
import { useSelector, useDispatch } from "react-redux";

import {
  increment,
  decrement
} from "../counterSlice";

function App() {

  const count = useSelector(
    state => state.counter.value
  );

  const dispatch = useDispatch();

  return (

    <div>

      <h1>Count: {count}</h1>

      <button
        onClick={() => dispatch(increment())}
      >
        Increment
      </button>

      <button
        onClick={() => dispatch(decrement())}
      >
        Decrement
      </button>

    </div>
  );
}

export default App;

```

6. What is Context API?

Context API is React's built-in feature used to share data globally without props drilling. It is simpler than Redux and suitable for small applications.

7. Context API Example

```
// UserContext.jsx

import { createContext } from "react";

export const UserContext =
  createContext();

// App.jsx

import { UserContext } from "../UserContext";

function App() {

  return (

    <UserContext.Provider
      value={"Vishvesh"}
    >

      <Child />

    </UserContext.Provider>
  );
}

// Child.jsx

import {
  useContext
} from "react";

import {
  UserContext
} from "../UserContext";

function Child() {

  const name =
    useContext(UserContext);

  return <h1>{name}</h1>;
}
```

8. Redux vs Context API

Feature	Redux	Context API
Type	External library	Built into React
Complexity	More complex	Simple
Performance	Better for large apps	Good for small apps
Boilerplate	More code	Less code
Best For	Large applications	Small applications
Middleware	Supported	Not available

9. When to Use Which?

Use Context API when:

- Small application

- Simple global data sharing
- Theme or authentication state

Use Redux when:

- Large application
- Complex state management
- Multiple components need shared state

10. Advantages of Redux

- Predictable state updates
- Centralized state management
- Easier debugging
- Better scalability
- DevTools support

11. Important Interview Definitions

Redux: Redux is a centralized state management library that stores application state in a global store.

Context API: Context API is React's built-in mechanism for sharing data globally without passing props manually.